

CSC6051 / MDS5112 – Image Processing and Computer Vision – Assignment 2

1 Programming Environment and Hardware Requirements

All tasks in this assignment must be completed using Python. Python is widely used in machine learning and deep learning, with rich library and framework support. You may choose either PyTorch or TensorFlow as your deep learning framework. While the example code in this document is primarily based on PyTorch, you may choose the framework according to your preference. **Note that if you opt for PyTorch, you must use version 2.0 or higher.**

Model training for this assignment requires GPU acceleration. All experiments are configured for a single GPU. You have three options for GPU resources: the university's GPU cluster, or the free GPU resources provided by Google Colab, a personal computer equipped with an NVIDIA GPU.

In university's GPU cluster, each student will be able to submit jobs requesting up to 4 CPU cores and 1 NVIDIA 2080 Ti GPU. **Please note that these computational resources are shared among students from this course as well as other courses, and jobs will be queued accordingly.**

Google Colab (<https://colab.research.google.com>) is a free cloud-based Jupyter notebook environment that provides GPU support. It offers several advantages, including no need for a local GPU, pre-installed deep learning libraries and frameworks, and the ability to write and run code directly in the browser. When using Google Colab, please be aware that GPU usage time is limited for each session, a Google account is required for login, and it's recommended to save your code and data to Google Drive for easy management and persistence.

If you choose to use a personal computer, ensure that your machine is equipped with an NVIDIA GPU, CUDA and cuDNN are correctly installed, and you have installed a GPU version of PyTorch or TensorFlow compatible with your CUDA version.

2 Task 1: Gaussian Filter (10 Points)

2.1 Background

This section aims to implement a simple Gaussian filter, considering only the case where the kernel size and standard deviation are the same in both the X and Y directions. Given an image I , a kernel size k and a standard deviation σ , the implemented function means to output the filtered image $\hat{I}$ according to k and σ .

2.2 TODO List

Implement the Gaussian filter function with the parameters kernel size and standard deviation. The kernel size is an odd number greater than 1, and the standard deviation is a non-negative number. Some examples are shown in Figure 1.

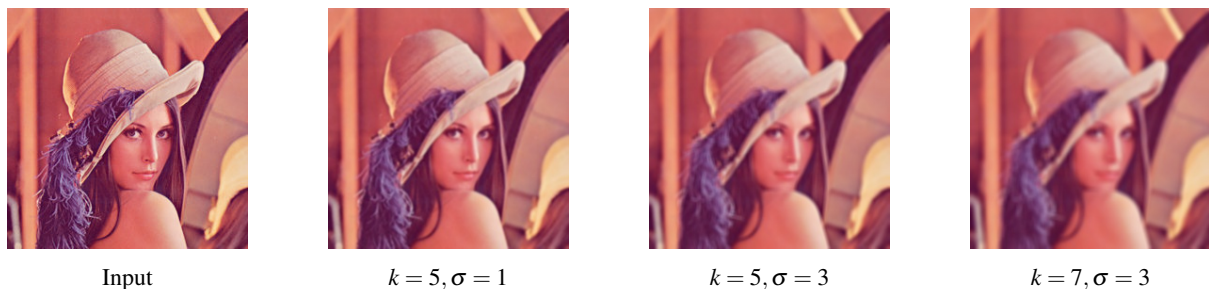


Figure 1: The visualization of our input and outputs.

2.3 Hints

1. You need to implement the Gaussian filter function in a pixel-wise way, referring to the formula from Lecture 4. It is **NOT** allowed to use any advanced image processing library APIs, such as `cv2.GaussianBlur()`, to perform the calculation directly. It will make the task too simple.
2. For edge pixels, such as the top-left corner pixel, some pixel values within the Gaussian kernel will be missing when calculating the result of the Gaussian filter. To handle this situation, you can choose not to process these edge values. For example, when the kernel size is 3, you can skip processing the pixels in the first and last rows, as well as the first and last columns of the input image, and directly start processing from the second element of the second row. Alternatively, you can pad the original input image, such as adding a border of zeros around the image when the kernel size is 3, before proceeding with the calculation. Other reasonable handling methods are also acceptable. No matter which method you choose for handling edge pixels, full marks will be awarded.

3 Task2: Local Histogram Equalization (20 points)

3.1 Background

Histogram equalization is a method in image processing of contrast adjustment using the image's histogram.

Ordinary histogram equalization uses the same transformation derived from the image histogram to transform all pixels. This works well when the distribution of pixel values is similar throughout the image. However, when the image contains regions that are significantly lighter or darker than most of the image, the contrast in those regions will not be sufficiently enhanced.

Local histogram equalization (Adaptive histogram equalization) improves on this by transforming each pixel with a transformation function derived from a neighborhood region. In this assignment, each pixel is transformed based on the histogram of a **square** surrounding the pixel.

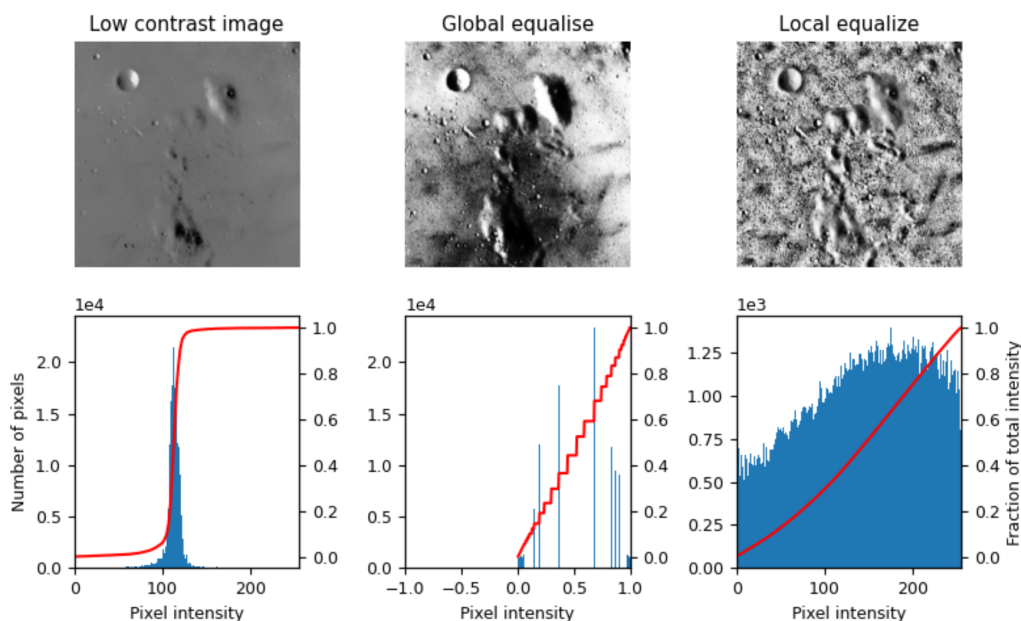


Figure 2: Comparison between the original and local histogram equalization.

3.2 TODO List

1. Implement the original histogram equalization method **histogram_equalization**. (5 points)
2. Implement the local histogram equalization method **local_histogram_equalization**. (10 points)
3. Save and compare the result from the local histogram equalization method to the original histogram equalization result. (5 points)

3.3 Hints

1. You should adjust the size of squares to get a natural result image.
2. You can refer to the wiki and the scikit-image library websites below.
3. The scikit-image library is **NOT** allowed to use directly. It will make the task too simple.

3.4 Reference

1. Histogram Equalization. https://en.wikipedia.org/wiki/Histogram_equalization
2. Adaptive Histogram Equalization. https://en.wikipedia.org/wiki/Adaptive_histogram_equalization
3. Local Histogram Equalization. https://scikit-image.org/docs/0.18.x/auto_examples/color_exposure/plot_local_equalize.html

4 Task3: Monocular Depth Estimation (70 points)

4.1 Background

This section deals with monocular depth estimation. Given a single RGB image, the goal is to predict a dense depth map, where each pixel indicates the distance from the camera to the corresponding 3D point in the scene.

In this task, you will first train a depth estimation model using the provided training pipeline and evaluate its performance on the test split. Next, you will investigate the effect of data scale by varying the number of training samples and the number of epochs. Finally, you will compare your baseline model with several recent foundation models for depth estimation and analyze the performance gap between small-scale supervised training and large-scale pretrained models. Before starting the task, it is important to distinguish between the following two concepts:

- **Depth.** Depth usually refers to the **metric distance** from the camera to the scene point. For example, if a pixel corresponds to a point that is 2.5 meters away from the camera, then its depth value is 2.5. In this assignment, the ScanNet dataset provides ground-truth metric depth maps.
- **Relative depth.** Relative depth does not aim to predict the exact metric distance. Instead, it only captures the ordinal relationship between pixels, i.e., which point is closer and which point is farther. For example, it may correctly predict that the chair is closer than the wall, even if the absolute distances are not numerically accurate. Many large pretrained depth models produce predictions that are more naturally interpreted as relative depth.

Since different models may predict depth values with different scales, it is often necessary to perform scale-shift alignment before evaluation. This aligns the predicted depth map with the ground-truth depth map and enables a fair comparison under the **Absolute Relative Error (AbsRel)** metric. The metric implementation and alignment procedure are provided in the reference code.

4.2 TODO List

1. **Baseline training using the provided model. (10 points)** Download the ScanNet dataset from this link. Then use the provided model and training pipeline to train a monocular depth estimation model on the ScanNet training split. The default experimental setting is:

- **32 epochs, max_train_samples = 10000**

The provided reference training code uses a ResNet50-based depth model and optimizes it with the SiLog loss.

2. **Evaluate the baseline model on the test split. (10 points)** After training, test your trained checkpoint on the validation/test split and report the final performance using **AbsRel**. You should use the provided testing script interface. In your report, include both:
 - quantitative evaluation results
 - qualitative depth visualization examples

The baseline testing pipeline loads the checkpoint, predicts depth, aligns the predictions with the ground truth using scale-shift alignment, and reports the final AbsRel score.

3. **Study the data scale effect. (20 points)** Repeat the training procedure under the following settings:

- 16 epochs, 20000 training samples
- 8 epochs, 40000 training samples
- 4 epochs, 80000 training samples
- 2 epochs, 160000 training samples

Compare the final performance of these four settings and analyze the trade-off between the number of training epochs and the number of training samples. Does using more data always improve performance? Is it better to train longer on a smaller subset, or shorter on a larger subset?

4. **Foundation model comparison. (30 points)** Evaluate and compare your baseline model against at least two 3D foundation models for depth estimation, for example VGGT, DA3, or other relevant large pretrained models. For this part, you are required to **implement the testing/evaluation pipeline** for the selected foundation models. The goal is to test the performance of these large models on ScanNet and compare them fairly against the baseline model using the same evaluation protocol.

4.3 Reference

1. Learn the Basics. PyTorch Tutorials. <https://pytorch.org/tutorials/beginner/basics/intro.html>
2. PyTorch Documentation. <https://pytorch.org/docs/stable/index.html>
3. VGGT: Visual Geometry Grounded Transformer. <https://arxiv.org/abs/2503.11651>
4. Depth Anything 3: Recovering the Visual Space from Any Views. <https://arxiv.org/abs/2511.10647>

5 Submission Guidelines

Your submission must include:

`report.pdf`: A comprehensive project report (maximum 6 pages including references) detailing your work. Only the provided LaTeX template (CVPR style) is allowed. The report should include: Detailed procedures with explanatory images (if needed), final pseudo-code, relevant mathematical theory, your thought process, thorough experiments and results analysis, and **a brief introduction to your code usage (Important!)**.

`code/`: A directory containing all your code for this assignment.

`checkpoints/`: Required checkpoints. For large files, provide a properly shared OneDrive link. Inaccessible links count as non-submission.

Grading: 60% report, 40% code. **Plagiarism is strictly prohibited and will be checked. Ensure your code runs without errors.**

Submission Process: Rename your folder as <your student ID>-<your name>-Asgn2, compress it to <your student ID>-<your name>-Asgn2.zip, and upload to the blackboard system.

Please read these guidelines carefully. Late submissions will incur penalties: 1 day late: -20 marks; 2 days: -40 marks; 3 days: -100 marks.